Habit Tracker Documentation

REST API Endpoints

`POST /api/auth/register`

Purpose: Register a new account **Authentication:** None

Headers:

```
Content-Type: application/json
```

Request:

```
{
  "name": "string",
  "email": "string",
  "password": "string"
}
```

Response:

```
{
"token":"string"
"user": { "id": "string", "name": "string", "email": "string"}
}
```

`POST /api/auth/login`

Purpose: Log into user's account **Authentication:** None

Headers

```
Content-Type: application/json
```

Request:

```
{
  "email": "string",
  "password": "string"
}
```

Response:

```
{
"token":"string"
"user": { "id": "string", "name": "string", "email": "string"}
}
```

`GET /api/auth/me`

Purpose: Log into user's account **Authentication:** Required

Headers

```
Content-Type: application/json
Authorisation: Bearer <token>
```

Response:

```
{
  "user": { "id": "string", "name": "string", "email": "string" }
}
```

`GET /api/habits`

Purpose: Fetch all of users habits. **Authentication:** Required

Headers

```
Content-Type: application/json
Authorisation: Bearer <token>
```

Response:

```
{
  "user": { "id": "string", "name": "string", "email": "string" }
}
```

`GET /api/habits`

Purpose: Fetch all of users habits. **Authentication:** Required

Headers

```
Content-Type: application/json
Authorisation: Bearer <token>
```

Response:

```
[
  {
    "userId": "string",
    "name": "string",
    "description": "string",
    "icon": "string"
  }
]
```

`POST /api/habits/add`

Purpose: add users habits. **Authentication:** Required

Headers

```
Content-Type: application/json
Authorisation: Bearer <token>
```

Request:

```
{
  "name": "string",
  "description": "string",
  "icon": "string"
}

**Response:**

```json
{
  "userId": "string",
  "name": "string",
  "description": "string",
  "icon": "string"
}
```
```

`PATCH /api/habits/update`

Purpose: Updating a habit. **Authentication:** Required

Headers

```
Content-Type: application/json
Authorisation: Bearer <token>
```

Request:

```
{

  "id": "string",
  "name": "string",
  "description": "string",
  "icon": "string"
}

**Response:**

```json
{
  "userId": "string",
  "name": "string",
  "description": "string",
  "icon": "string"
}
```

`DELETE /api/habits/delete`

Purpose: Deleting a habit. **Authentication:** Required

Headers

```
Content-Type: application/json
Authorisation: Bearer <token>
```

Request:

```
{

  "habitId": "string",

}
```
```

````
**Response:**

```json
{
  "success": "boolean"
}
````

`GET /api/dailyEntry/:date`

Purpose: Get todays daily entries. **Authentication:** Required

Headers

```
Content-Type: application/json
Authorisation: Bearer <token>
```

Parameter::

````
{

  `date - dd/mm/yyyy`

}

**Response:**

```json
{
  "userId": "string",
  "date": "string",
  "habits": [
  {
  "habit": "string",
  "completed": "boolean" `
  }
  ]
}
````

`PATCH /api/dailyEntry/update`

Purpose: Get todays daily entries. **Authentication:** Required

Headers

```
Content-Type: application/json
Authorisation: Bearer <token>
```

Parameter::

```
{

  "habitId" : "string",
  "date": "string",
  "completed": "boolean"

}

**Response:**

```json
{
  "userId": "string",
  "date": "string",
  "habits": [
  {
  "habit": "string",
  "completed": "boolean" `
  }
  ]
}
```

Dependencies

Runtime Dependencies

Package	Version	License	Purpose
bcrypt	^6.0.0	MIT	Hashing passwords
dotenv	^17.4.2	MIT	Environment variables
express	^5.2.1	MIT	Web framework for node js
jsonwebtoken	^9.0.3	MIT	Implentation of json webtokens
mongoose	^9.9.1	MIT	MongoDB ODM

Dev dependencies

Package	Version	License	Purpose
@tailwindcss/cli	^4.3.3	MIT	CLI for tailwind
```

Package	Version	License	Purpose
tailwindcss	^4.3.3	MIT	Tailwind dev
concurrently	^10.0.4	MIT	Run multiple commands

Credentials

An example of the credentials is in `.env.example`

```
MONGODB_URI=your-mongdb-uri
JWT_SECRET=random-secret
```

Feedback

Feedback can be sent to my email address jesline.ttt@outlook.com

Process

1. Feedback is logged and categorised and assigned a priority level.
2. Urgent feedback is fast tracked.
3. Feedback is published in release notes.
4. Changes are reviewed and tested.
5. Feedback policy is reviewed quartely

Guidelines and Policies

- Privacy Policy
- Terms of Service
- Copyright

Code Commit Procedure

1. Create a new branch.
2. Make changes and test locally.
3. Commit changes to new branch.
4. Create a pull request to dev or main branch
5. Review an merge changes, handle code conflicts.
6. CI/CD workflow is handled by Vercel.
7. Ensure deployment was successful by checking Vercel logd.
8. Delete feature branch.

Best practices

- Comitting frequently
- Using clear messages

- Participating in reviews
- Maintaining tests
- Updating documentation

Documentation

Documentation can be found here: check out the Doucmentation